

An AI-Powered Technical Debt Analytics Dashboard for Agile Small-Scale Software Teams

Project Proposal

Group – 16

Project ID – 7

CS3203 – Software Engineering Project

Mentor: Mr. Anju Chamantha

Name	Index number
Nethmini R.M.N.	230432G
Nethsara U.D.K.C.	230435T
Pabasara H.H.J.	230451M

Date: 5th July, 2026

Table of Contents

1. Overview of the Project	1
2. Objectives of the Project.....	1
3. The Need for the Project	1
4. Scope of the Project	2
4.1 Core Functionalities:.....	2
4.2 Out of Scope:	2
5. Deliverables	2
6. Overview of Existing Systems and Technology	3
6.1 Existing Techniques	3
6.2 Existing systems and tools.....	3
SonarQube.....	3
CodeScene.....	3
CodeRabbit	3
6.3 Proposed System Implementation.....	3
7. References	4

List of Tables

Table 1: Proposed Technology Stack and Resources for System Components	4
--	---

1. Overview of the Project

The project proposes an AI-Powered Technical Debt Analytics Dashboard for small-scale agile software teams; delivered as multi-tenant web-based SaaS platform. The platform allows the teams to identify, analyze and prioritize technical debt. Teams connect their GitHub/GitLab repositories, and the system will analyze the source code, code comments, and commit messages of the selected repository to identify different types of technical debt using static analysis and machine learning. It will output a code health score, categorize the identified technical debt and prioritize the issues. Unlike existing tools, the platform consists of cross repository analysis, organization configurable profiles. This allows developers to understand the current condition of their codebase and identify the areas that require immediate attention.

2. Objectives of the Project

- Analyze direct impacts of technical debt to the velocity of small-scale agile teams.
- Design and implement a secure, multi-tenant web architecture for analyzing code repositories.
- Provide a centralized, interactive dashboard visualizing technical debt metrics in a single interface.
- Develop and train an NLP-driven Machine Learning module capable of classifying source code comments and commit messages into debt categories.
- Evaluate the accuracy of ML-driven bug prediction algorithms compared to traditional, rule based static code analysis.

3. The Need for the Project

Small and medium scale agile software teams work under tight deadlines and limited resources often leading to bad smells in code. These firms often lack Senior software engineers' reviews, so more bad smell produced by less experienced developers often go unnoticed. Recent research found that the developers spent roughly 23% of their time on technical debt which will be a major pain point for small agile teams due to lack of staff on manual code reviews [1]. Combined with all, the overall project velocity reduces unnoticeably.

Yet there are numerous tools for code reviewing, small agile teams struggle to adapt them as they are built for large organizations which have dedicated teams for configuration and code reviews. It is rarely found in small agile teams with diverse engineering backgrounds; Therefore, these systems make high overhead in configuration and adapting the software. On the other hand, above systems mostly rely on static deterministic rules, resulting in hundreds of issues making it unsuitable for small agile teams. Rather than having unnecessary noise and complex configurations, these teams prefer simple tools which only prioritize the most critical issues that need immediate fixing. In agile principal teams prefer adding new features in each increment, therefore fixing low priority technical debt has no visible customer value.

4. Scope of the Project

4.1 Core Functionalities:

- Connect repository with GitHub App/ Integrate with GitHub Actions workflows/ or paste a URL directly.
- Cross repository dependency analytics across workspace repositories.
- Run analysis with asynchronous worker queues to keep UI responsive.
- Support silent checks (silent check is a background check automatically triggered on every push or pull request) to auto refresh dashboard.
- Detection of security related technical debt (security keyword SATD, hardcoded secrets, SQL string concatenation, dangerous eval/exec).
- Provide a dashboard with overall code health with interactive file tree view to show hotspots in code.
- Produce prioritized list of “fix first” issues using organization configurable weighting profiles.

4.2 Out of Scope:

- Fully automatic code refactoring service
- Support for proprietary/on-premise distributed version control system beyond GitHub/GitLab
- Integration with Project management tools such as Jira/Trello
- Full vulnerability scanning (SAST/DAST, CVE/dependency auditing, penetration testing).

5. Deliverables

- **A web-based software system (SaaS):**
A multi-tenant system where a user connects their GitHub/ GitLab repository to receive a health report & Commit graph with refreshable Dashboard
- **An interactive Graphical User Interface (GUI):**
A user-friendly and interactive dashboard which displays the debt analysis outcomes clearly to identify easily using prioritizing according to user preferences.
- **A Machine Learning Model:**
A Python based trained machine learning model automatically scans code comments and commits messages for debt classification. (documented precision/recall metrics for debt classification)
- **System architecture documentation:**
Complete technical documentation explained the interaction between the backend and the ML service.
- **A project report:**
A final detailed report which validates the tool’s utility by analyzing the real-world existing repositories.

6. Overview of Existing Systems and Technology

6.1 Existing Techniques

Static code analysis is a fundamental software quality assurance technique that involves systematically inspecting source code without executing it aiming to detect bugs, vulnerabilities, code smells and compliance issues and it helps minimize and address technical debt [2]. To identify and prioritize the identified technical debt, some tools use Behavioral Code Analysis. Behavioral Code Analysis identifies patterns in how a development organization interacts with the code base they are building. This analysis is used to prioritize technical debt, detect implicit dependencies that are invisible in the code itself and measure organizational factors like knowledge gaps and support on and off boarding [3]. With the advancement of Artificial Intelligence, tools use AI-powered static code analysis to identify problem areas. Also, they analyze historical data, learning from past coding patterns and their outcomes so that the analysis can alert teams to potential debt-ridden areas in codebase, even before they become a significant problem [4].

6.2 Existing systems and tools

SonarQube [5]

SonarQube is a self-managed, enterprise-grade and static analysis based continuous code quality and static application security testing platform that supports more than thirty programming languages [6]. However, it requires considerable configuration effort for small development teams and generates a large number of findings that require manual filtering.

CodeScene [7]

CodeScene is a platform which uses behavioral code analysis to identify and prioritize technical debt of the software products and it outputs prioritized hotspot maps, numeric code health scores, financial cost impact trends, and automated delivery risk classifications [8]. Even though it provides powerful organizational insights, it cannot output actionable results for codebases of a small-scale team.

CodeRabbit [9]

CodeRabbit is an AI-powered code review platform that analyzes every code change throughout the pull request lifecycle. It provides automated reviews, identify potential bugs, security vulnerabilities, code quality issues and suggests possible fixes [4]. As it scans the code only at each increment, it does not allow to detect where older technical debt exists or to track long-term repository health across the entire project lifecycle

6.3 Proposed System Implementation

The proposed solution combines lightweight static code analysis with machine learning techniques to provide actionable technical debt prioritization for small-scale agile software teams instead of generating extensive lists of code quality issues. The source code is analyzed using static analysis to extract software quality metrics while code comments and commit messages are analyzed in parallel using NLP-based Self-Admitted Technical Debt (SATD) classifier. The extracted software metrics are used by a bug prediction model to estimate the likelihood of future defects. Then, the outputs of the static analysis, SATD classifier and bug prediction model are integrated into a configurable technical debt score that ranks source files according to their refactoring priority. The results are displayed on the interactive dashboard.

Table 1: Proposed Technology Stack and Resources for System Components

Component	Task	Technology/ Resource
Frontend	Analytics dashboard	Next.js [10]
Backend	Repository integration and API	FastAPI [11]
Static Analysis	Extract software metrics	Lizard [12]
ML Module	SATD detection and classification	Python [13], Scikit-Learn [14], SATD Dataset [15]
	Bug proneness prediction	Python [13], Scikit-Learn [14], GHPR dataset [16], NASA PROMISE dataset [17]
Database	Store analysis results	PostgreSQL [18]

7. References

- [1] T. Besker, A. Martini and J. Bosch, "Technical Debt Cripples Software Developer Productivity," 2018.
- [2] Sonar, "Static Code Analysis Using SonarQube Community Build: A Step-by-Step Guide," Sonar, [Online]. Available: <https://www.sonarsource.com/resources/library/static-code-analysis-using-sonarqube/>. [Accessed 1 July 2026].
- [3] Adam Tornhill, "Better than silver bullets: A milestone for behavioral code analysis," CodeScene, [Online]. Available: <https://codescene.com/blog/validation-for-behavioral-code-analysis/>. [Accessed 1 July 2026].
- [4] CodeRabbit, "Reduce Tech Debt: AI's Role in Efficient Coding," CodeRabbit, [Online]. Available: <https://www.coderabbit.ai/blog/reduce-tech-debt-ai-s-role-in-efficient-coding>. [Accessed 1 July 2026].
- [5] Sonar, "SonarQube - Code verification for the AI era," Sonar. [Online]. [Accessed 1 July 2026].
- [6] Sonar, "Static Code Analysis Using SonarQube Community Build : A Step-by-Step Guide," 2026. [Online]. Available: <https://www.sonarsource.com/resources/library/static-code-analysis-using-sonarqube/>. [Accessed 1 July 2026].
- [7] CodeScene, "Scale AI Coding Safely and Mitigate Technical Debt," CodeScene. [Online]. [Accessed 1 July 2026].
- [8] CodeScene, "Behavioral Code Analysis in Practice," CodeScene, 2023.
- [9] CodeRabbit, "AI Code Reviews | CodeRabbit," CodeRabbit, [Online]. Available: <https://www.coderabbit.ai/>. [Accessed 2 July 2026].

- [10] Vercel, "Next.js by Vercel - The React Framework," Vercel, [Online]. Available: <https://nextjs.org/>. [Accessed 2 July 2026].
- [11] FastAPI, "FastAPI," [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed 2 July 2026].
- [12] T. Yin, "Lizard," [Online]. Available: <https://github.com/terryyin/lizard/blob/master/README.rst>. [Accessed 2 July 2026].
- [13] Python Software Foundation, "Welcome to Python.org," Python Software Foundation. [Online]. [Accessed 2 July 2026].
- [14] scikit-learn, "scikit-learn: machine learning in python," scikit-learn, [Online]. Available: <https://scikit-learn.org/stable/>. [Accessed 2 July 2026].
- [15] Y. Li, M. Soliman and P. Avgeriou, "Automatic identification of self-admitted technical debt from four different sources," *Empirical Software Engineering*, vol. 28, no. 65, 2023.
- [16] J. Xu, F. Wang and J. Ai, "Defect Prediction With Semantics and Context Features of Codes Based on Graph Representation Learning," *IEEE Transactions on Reliability*, 2021.
- [17] ApoorvaKrisna, "NASA Promise Dataset Repository," 2020. [Online]. Available: <https://github.com/ApoorvaKrisna/NASA-promise-dataset-repository>. [Accessed 4 July 2026].
- [18] The PostgreSQL Global Development Group, "PostgreSQL: The world's most advanced open source database," The PostgreSQL Global Development Group, 2026. [Online]. Available: <https://www.postgresql.org/>. [Accessed 1 July 2026].
- [19] OSWAP Foundation, "Static Code Analysis," [Online]. Available: https://owasp.org/www-community/controls/Static_Code_Analysis. [Accessed 30 June 2026].
- [20] CodeScene, "Behavioral Code Analysis in Practice," CodeScene, 2023.
- [21] Codacy, "Official information about Codacy," 27 May 2026. [Online]. Available: <https://www.codacy.com/ai-info#basic-information>. [Accessed 1 July 2026].
- [22] Codacy, "Code Qyality for AI-Assisted Engineering Teams | Codacy," Codacy. [Online]. [Accessed 1 July 2026].